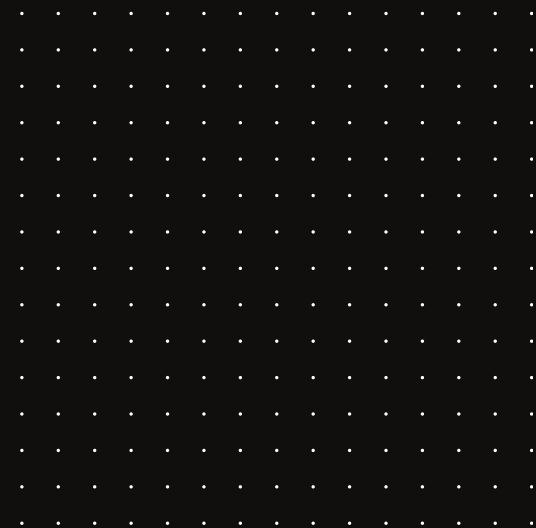


Diseño de Arquitectura de Solución

Sistema de Banca por Internet

Versión 1.0



Junio 2026

SOFTWARE ARCHITECT:
José E. Chiroy Aifán

Tabla de Contenido

01	Resumen Ejecutivo	3
02	Consideraciones Normativas	4
03	Decisiones Arquitectónicas	5
04	C4 Nivel 1 — Contexto	8
05	C4 Nivel 2 — Contenedores	9
06	C4 Nivel 3 — Componentes	11
07	Arquitectura de Autenticación	15
08	Onboarding y Biometría	16
09	Frontend: SPA y Móvil	17
10	Auditoría — Event Sourcing	17
11	Estrategia de Caché	18
12	Alta Disponibilidad y DR	19
13	Infraestructura Cloud AWS	20
14	Monitoreo y Observabilidad	21

01 Resumen Ejecutivo

BP requiere un sistema de banca por internet multicanal que permita a sus clientes consultar el histórico de movimientos, realizar transferencias entre cuentas propias e interbancarias, y efectuar pagos. El sistema debe operar bajo los más altos estándares de seguridad del sector financiero internacional.

La solución propuesta se basa en una **arquitectura de microservicios desacoplada** desplegada en **AWS**, con dos aplicaciones cliente — una SPA en **Angular** y una aplicación móvil multiplataforma en **Flutter** —, una capa de integración compuesta por **API Gateway + WAF**, un servidor de identidad **OAuth 2.0 / OIDC** (Keycloak), y cinco microservicios de negocio. El sistema emplea **Event Sourcing** para auditoría inmutable, **Cache-Aside** con Redis para rendimiento, y **reconocimiento facial** (AWS Rekognition) en el proceso de Onboarding.

La infraestructura garantiza Alta Disponibilidad (Multi-AZ), Tolerancia a Fallos (Circuit Breaker, Retry, Timeout), Recuperación ante Desastres (RPO < 1h / RTO < 4h con región secundaria sa-east-1), y Monitoreo integral mediante AWS CloudWatch y X-Ray.

02Consideraciones Normativas y Regulatorias

La arquitectura incorpora los siguientes marcos regulatorios y estándares de seguridad aplicables a entidades financieras:

Regulación / Estándar	Descripción	Impacto Arquitectónico
PCI DSS v4.0	Estándar de seguridad para datos de tarjetas de pago	TLS 1.3 obligatorio, tokenización de tarjetas, redes segmentadas, logs de acceso, escaneo de vulnerabilidades trimestral
ISO 27001 / 27002	Sistema de Gestión de Seguridad de la Información	WAF + IDS/IPS, gestión de parches, pruebas de penetración anuales, clasificación y cifrado de activos de información
FAPI (Financial-grade API)	Perfil de seguridad OAuth 2.0 para servicios financieros	mTLS para comunicación entre servicios, DPoP token binding, PKCE obligatorio, tokens de corta duración
OWASP Top 10	Vulnerabilidades críticas en aplicaciones web	Validación de entradas, CSP headers, CSRF tokens, sanitización XSS, gestión segura de dependencias

03 Decisiones Arquitectónicas (ADRs)

Cada decisión fue evaluada frente a alternativas concretas. Se exponen mínimo dos justificaciones teóricas por decisión.

ADR-01 Framework Móvil Multiplataforma

OPCIONES EVALUADAS

React Native - Flutter

SELECCIÓN

Flutter (Dart)

PATRÓN APLICADO

BLoC / Provider

J1 — Rendimiento nativo: Flutter compila a código ARM nativo (AOT) eliminando el bridge JavaScript de React Native, reduciendo latencia crítica en operaciones bancarias y animaciones de biometría. El motor Skia/Impeller garantiza 60/120 fps consistentes.

J2 — Seguridad bancaria: El paquete local_auth expone APIs nativas de biometría (Face ID, Touch ID, huella Android) y flutter_secure_storage usa iOS Keychain / Android Keystore para tokens, fundamentales en entornos financieros regulados.

ADR-02 Framework SPA

OPCIONES EVALUADAS

React - Angular

SELECCIÓN

Angular 18

PATRÓN APLICADO

Módulos + Lazy Loading

J1 — Arquitectura empresarial: Angular es un framework MVC opinado con TypeScript nativo, inyección de dependencias y módulos, lo que reduce inconsistencias en equipos grandes y facilita auditorías de código, un requisito implícito en entornos financieros regulados.

J2 — Seguridad integrada: Angular sanitiza automáticamente el DOM contra XSS, los HttpInterceptors adjuntan tokens JWT a todas las peticiones de forma centralizada, y el módulo de Route Guards controla el acceso por roles sin lógica dispersa.

ADR-03 Flujo OAuth 2.0 Recomendado

OPCIONES EVALUADAS
Implicit · Client Credentials -
Auth Code + PKCE

SELECCIÓN
Authorization Code + PKCE

RFC
RFC 6749 / RFC 7636 / RFC
9700

J1 — Implicit Flow deprecado: El RFC 9700 (OAuth 2.0 Security BCP) deprecó el Implicit Flow por exponer el access_token directamente en la URL/fragment, vulnerable a historial de navegador y referrer headers. PKCE previene ataques de interceptación del código de autorización sin necesitar client_secret.

J2 — Estándar para clientes públicos: Authorization Code + PKCE es el flujo obligatorio para SPAs y apps móviles según el BCP de OAuth 2.0 y el perfil FAPI 2.0, que exige su uso en aplicaciones financieras de alto riesgo. Permite refresh tokens rotativos para sesiones largas con mínima superficie de ataque.

ADR-04 Proveedor de Identidad (IdP)

OPCIONES EVALUADAS
Azure AD B2C - Keycloak

SELECCIÓN
Keycloak 24 (self-hosted)

DEPLOY
EKS — HA Multi-AZ

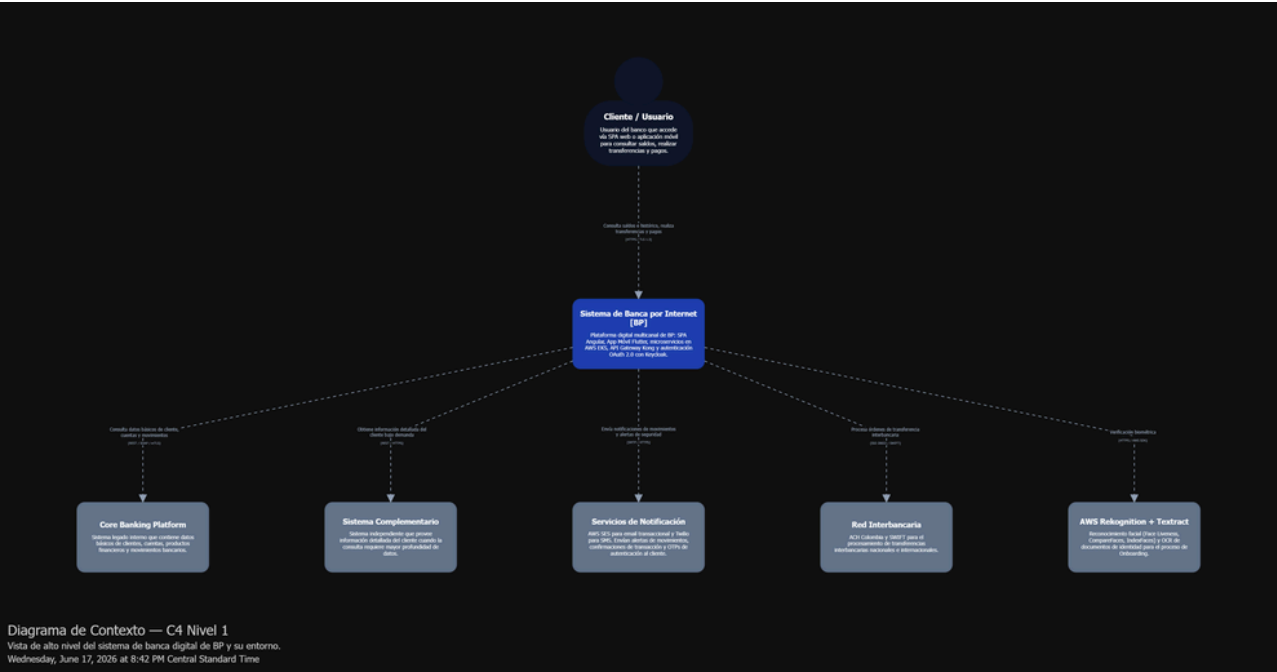
J1 — Soberanía de datos: Keycloak self-hosted en AWS garantiza que los datos de identidad de clientes nunca salen de la infraestructura controlada por BP, cumpliendo a la ley local de protección de datos aplicable y la circular de la Superfinanciera sobre tercerización de servicios críticos.

J2 — Soporte FAPI: Keycloak soporta nativamente FAPI 1.0/2.0, incluyendo mTLS, DPoP token binding y políticas de autenticación paso a paso (step-up), sin licencias adicionales. Su modelo de extensión permite integrar el flujo de reconocimiento facial del Onboarding como un Authentication SPI personalizado.

No.	Decisión	Selección	Justificación Clave
ADR-05	Reconocimiento Facial	AWS Rekognition	Integración nativa con el ecosistema AWS (S3, KMS, Lambda); certificaciones SOC2 Type II y PCI DSS Level 1 nativas; <i>Liveness Detection</i> anti-spoofing certificado
ADR-06	Patrón de Caché	Cache-Aside (Lazy)	Evita cold-start masivo; permite invalidación explícita desde el servicio cuando el Core actualiza datos desde canales externos; el consumidor controla coherencia
ADR-07	Patrón de Auditoría	Event Sourcing	Trazabilidad completa e inmutable requerida por regulación; auditoría asíncrona (SQS) no impacta latencia de transacciones principales; permite reconstrucción de estado completo
ADR-08	Notificaciones	AWS SES + Twilio SMS	AWS SES: alta entregabilidad email a \$0.10/1000; Twilio: cobertura superior SMS en LATAM con SLAs enterprise; 2 proveedores garantizan redundancia por normativa
ADR-09	API Gateway	AWS API GW + Kong	AWS API GW integra nativamente WAF, CloudWatch y Cognito; Kong añade rate limiting avanzado, plugins OIDC y developer portal enterprise desacoplados de la infraestructura cloud
ADR-10	Plataforma Cloud	AWS (us-east-1 + sa-east-1)	Mayor penetración en sector financiero LATAM; región sa-east-1 (Brasil) con baja latencia; servicios especializados GuardDuty, FinSpace, Macie para compliance financiero; certificación PCI DSS Level 1 nativa

04C4 Nivel 1 — Diagrama de Contexto

Para usuarios no técnicos. Muestra el sistema, sus actores clave y los sistemas externos con los que interactúa.

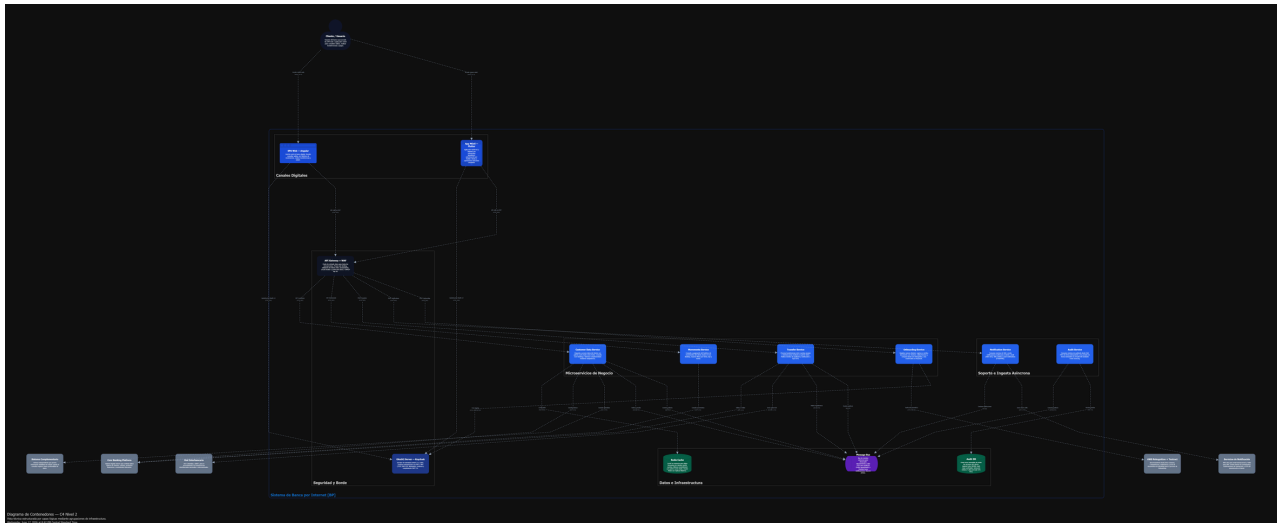


Elementos del Diagrama

Elemento	Tipo	Descripción	Protocolo
Cliente / Usuario	Persona	Usuario final del banco, accede vía SPA o App Móvil	HTTPS / TLS 1.3
Sistema Banca Internet [BP]	Sistema Interno	Plataforma multicanal: SPA Angular, App. Flutter, microservicios, infraestructura AWS	—
Core Banking Platform	Sistema Interno Legado	Plataforma core con información básica de clientes, cuentas, productos y movimientos	REST / SOAP / mTLS
Sistema Complementario	Sistema Externo	Complementa información detallada del cliente cuando se requiere mayor profundidad	REST / HTTPS
Servicios de Notificación	Sistema Externo (*2)	AWS SES para email transaccional y Twilio para SMS — doble canal por normativa	SMTP / HTTPS
Red Interbancaria	Sistema Externo	Red Interbancaria Local (ACH / Cámara de Compensación Automatizada)	ISO 20022 / SWIFT

05 C4 Nivel 2 — Diagrama de Contenedores

Para audiencia técnica. Muestra todas las aplicaciones, servicios, bases de datos y buses de mensajería dentro del sistema BP.



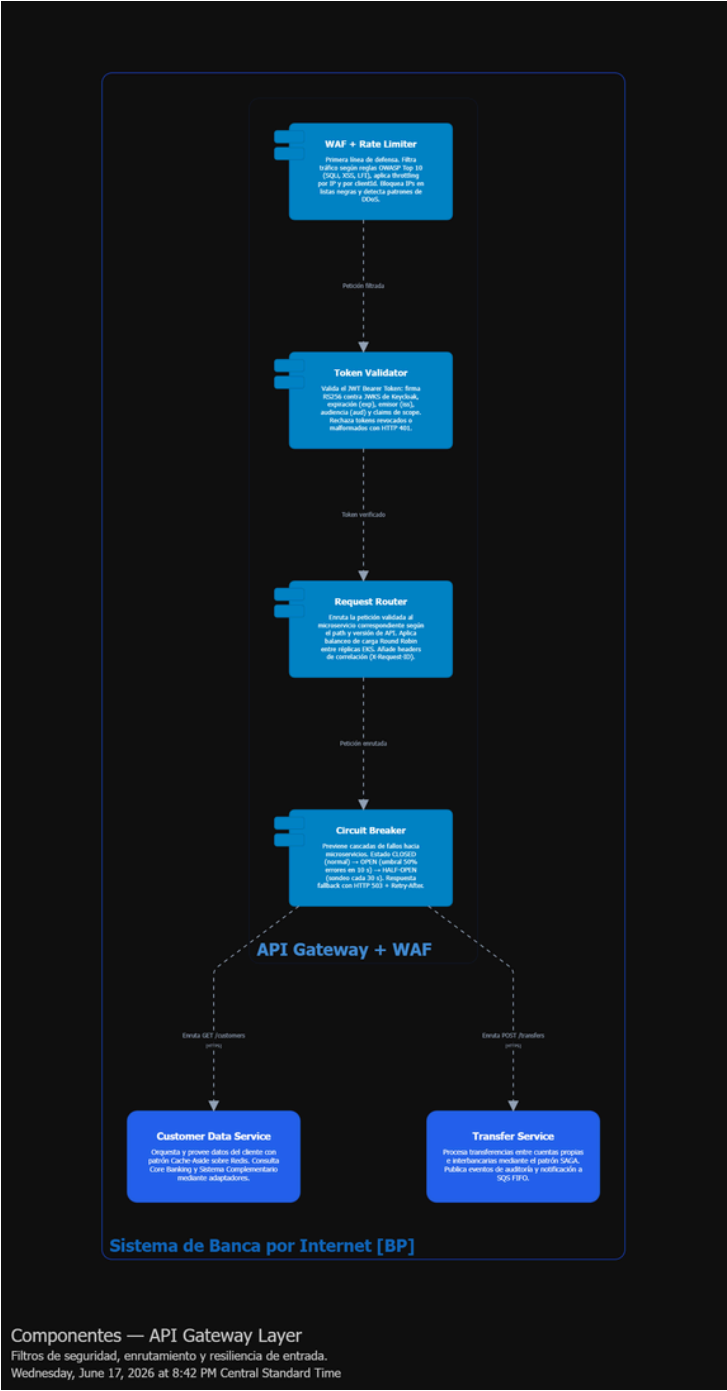
Contenedores Principales

Contenedor	Tecnología	Responsabilidad Principal
SPA Web Angular	Angular 18 · TypeScript · NgRx	Interfaz web de banca: saldos, movimientos, transferencias, pagos
App Móvil Flutter	Flutter 3 · Dart · BLoC	App iOS/Android con biometría, onboarding facial y operaciones bancarias
Keycloak (OAuth2/OIDC)	Keycloak 24 · Java · EKS HA	Emisión de tokens JWT, PKCE, MFA, gestión de sesiones, FAPI compliance
API Gateway + WAF	AWS API GW + Kong + AWS WAF	Enrutamiento, autenticación de tokens, rate limiting, protección DDoS, logging
Customer Data Service	Java 21 · Spring Boot 3 · EKS	Consulta y orquestación de datos del cliente con Cache-Aside pattern
Transfer Service	Java 21 · Spring Boot 3 · EKS	Procesamiento de transferencias propias e interbancarias, publicación de eventos de auditoría
Onboarding Service	Python 3.12 · AWS Lambda	Enrolamiento de nuevos clientes con reconocimiento facial vía AWS Rekognition
Audit Service + DB	Java · PostgreSQL · RDS Multi-AZ	Persistencia inmutable de todos los eventos del sistema via Event Sourcing / WORM
Redis Cache	AWS ElastiCache Redis 7 — Cluster	Cache-Aside para datos de cliente frecuentes, sesiones, rate limiting
Message Bus	AWS SQS FIFO + SNS	Desacoplamiento asíncrono entre servicios: eventos de auditoría, notificaciones

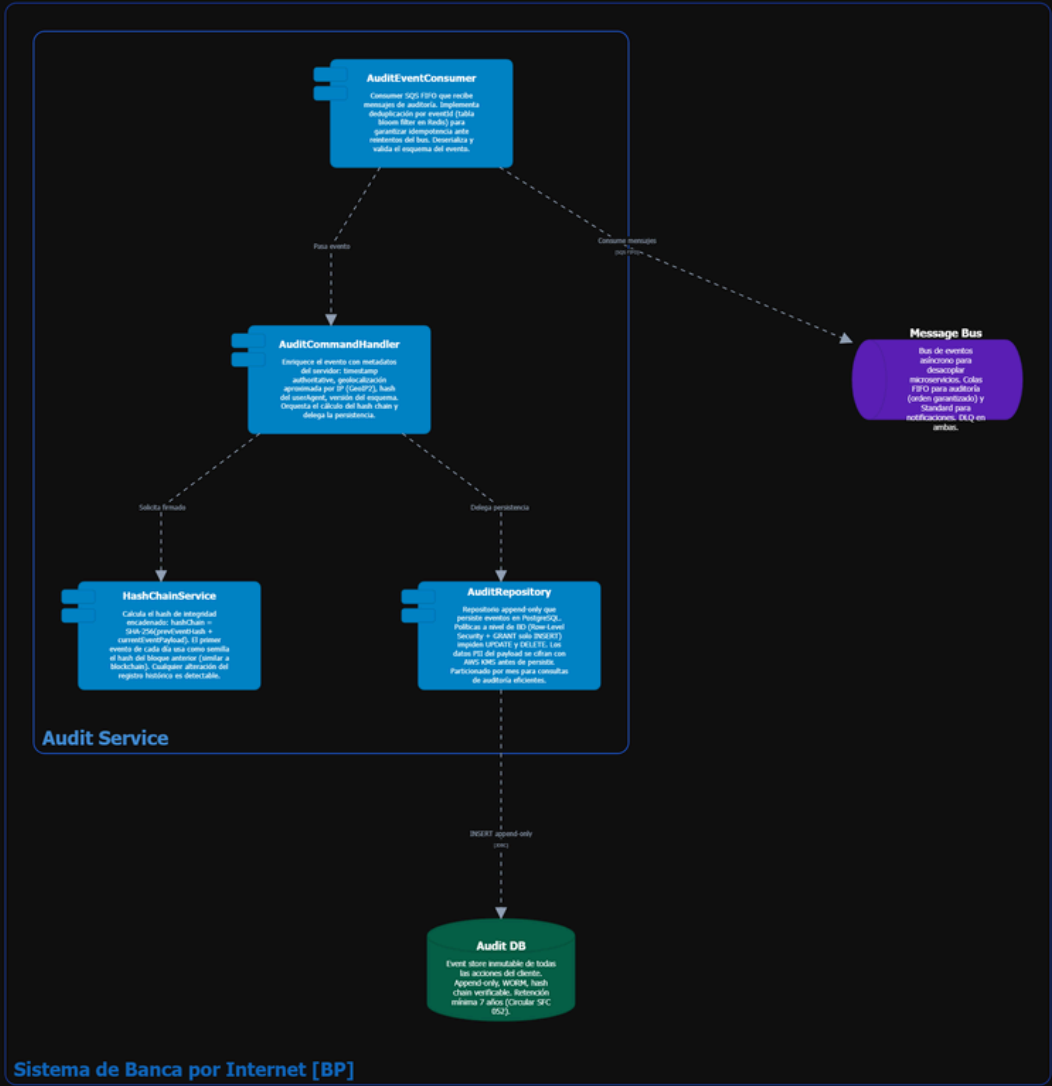
06 C4 Nivel 3 — Diagrama de Componentes

Mayor detalle técnico. Muestra los componentes internos de los servicios clave: Customer Data Service, Transfer Service y Audit Service, con sus patrones y protocolos.

API Gateway



Audit Service

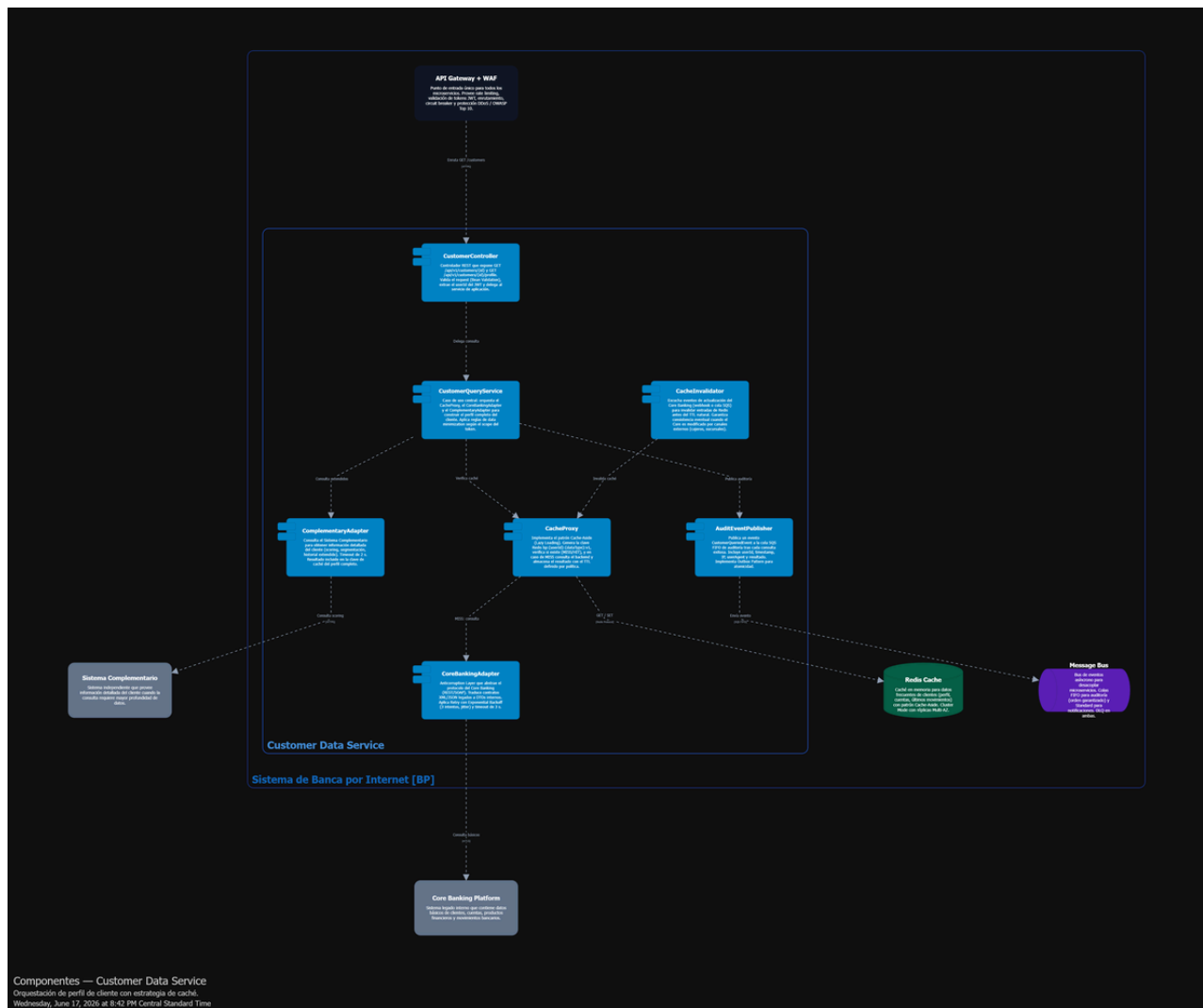


Componentes — Audit Service

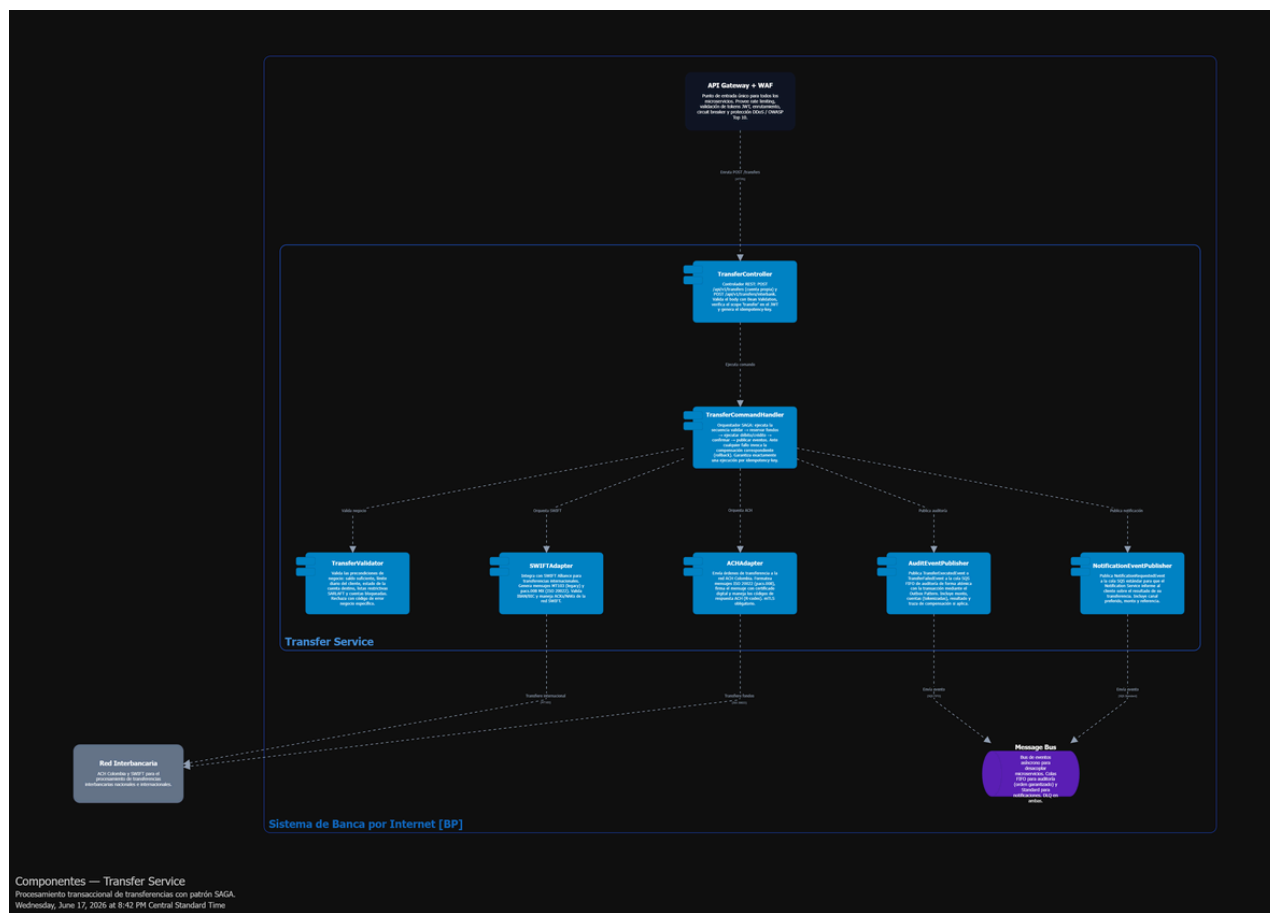
Ingesta inmutable de eventos con encadenamiento criptográfico.

Wednesday, June 17, 2026 at 8:42 PM Central Standard Time

Customer Data Service



Transfer Service



07 Arquitectura de Autenticación — OAuth 2.0

Flujo Recomendado: Authorization Code + PKCE

Secuencia del flujo para SPA y App Móvil:

- 1. La app genera un **code_verifier** aleatorio y calcula su **code_challenge** (SHA-256).
- 2. Redirige al Authorization Endpoint de Keycloak con **response_type=code&code_challenge=...&code_challenge_method=S256**.
- 3. El usuario se autentica (password + MFA). Keycloak retorna un **authorization_code** de corta vida (60 s).
- 4. La app intercambia el código por tokens enviando el **code_verifier**. Keycloak verifica el hash → entrega **access_token** (15 min) + **refresh_token** rotativo (8 h).
- 5. Cada petición al API Gateway incluye el **Authorization: Bearer <access_token>**. El Token Validator valida firma RS256 y claims.

Métodos MFA Soportados

TOTP: Google Authenticator / Microsoft Authenticator (RFC 6238)
Biometría nativa: Huella / Face ID (post-onboarding)
OTP SMS: Twilio Verify (canal backup)
WebAuthn (FIDO2): Hardware keys como Yubikey para usuarios avanzados

Configuración de Tokens

Token	TTL	Notas
Access Token	15 min	RS256, sin estado, validado localmente en API GW
Refresh Token	8 h (rotativo)	Se invalida en cada uso, nuevo RT emitido
Auth Code	60 s	Un solo uso; PKCE obligatorio

08 Onboarding y Arquitectura Biométrica

Flujo de Enrolamiento (Onboarding)



Autenticación Post-Onboarding

El nuevo usuario puede autenticarse en sesiones posteriores con cualquiera de estos métodos:

- **Facial biométrico:** SearchFacesByImage en colección de AWS Rekognition, umbral 99% de confianza
- **Huella dactilar:** local_auth de Flutter usa TouchID/Fingerprint nativo sin enviar datos al servidor
- **Usuario + clave + OTP:** Flujo estándar para acceso web con TOTP o SMS backup

Herramientas Recomendadas

Función	Herramienta
Reconocimiento facial	AWS Rekognition
Liveness detection	AWS Rekognition Face Liveness
Biometría mobile	Flutter local_auth (iOS/Android)
OCR de documento	AWS Textract
Verificación identidad	Onfido / Jumio (alternativa SaaS)

09 Arquitectura Frontend — SPA y Aplicación Móvil

Flujo de Enrolamiento (Onboarding)

Características clave: TypeScript nativo, módulos lazy-loaded, HttpInterceptors para JWT, Route Guards por roles, Angular CDK para accesibilidad (WCAG 2.1).

Estado: NgRx (Redux pattern) para manejo de estado global y caché local de datos del usuario.

Seguridad: Sanitización automática XSS, CSP headers vía meta-tag, CORS strict en API GW, SameSite Cookies para CSRF.

App Móvil — Flutter 3

Características clave: Un codebase para iOS y Android, BLoC pattern para estado reactivo, Material Design adaptado a branding BP.

Seguridad móvil: Certificate pinning (TrustKit/dio), detección de jailbreak/root (flutter_jailbreak_detection), almacenamiento en Keychain/Keystore (flutter_secure_storage).

Acceso sin conectividad: Consulta de últimos movimientos en caché local cifrada (Hive + AES-256).

10 Auditoría — Event Sourcing

Todas las acciones del cliente generan un evento de auditoría inmutable publicado en SQS FIFO y persistido por el Audit Service en PostgreSQL con restricciones WORM (Write Once Read Many). Esta arquitectura satisface los requerimientos de la Circular SFC 052 y PCI DSS.

Autenticación Post-Onboarding

```
{
  "eventId": "uuid-v7",
  "timestamp": "2026-06-17T10:34:56.123Z",
  "userId": "obfuscated-id",
  "action": "TRANSFER_EXECUTED",
  "channel": "MOBILE_APP",
  "ipAddress": "[cifrado KMS]",
  "deviceFingerprint": "sha256-hash",
  "payload": { "amount": 500000, "currency": "COP", "destAccount": "[tokenized]" },
  "result": "SUCCESS",
  "hashChain": "sha256(prevHash + eventData)"
}
```

El **hashChain** encadena criptográficamente cada evento con el anterior (similar a blockchain), haciendo detectable cualquier alteración. Los datos de PII son cifrados con KMS antes de ser persistidos.

11 Estrategia de Caché — Cache-Aside Pattern

Flujo Cache-Aside

Cache HIT (dato fresco)

Request → Service → Check Redis → FOUND → Return cached data

Cache MISS (primera vez / expirado)

Request → Service → Check Redis → NOT FOUND → Fetch Core Banking → Store Redis (TTL) → Return data

TTL por Tipo de Dato

Tipo de Dato	TTL
Perfil de cliente	5 min
Listado de cuentas	3 min
Últimos movimientos	2 min
Tasas / límites	15 min

Componentes que Interactúan

El patrón implementa los siguientes componentes relacionados con la persistencia para clientes frecuentes:

- **CacheProxy:** intercepta el CustomerQueryService antes de contactar el Core
- **Redis ElastiCache:** almacén de caché en memoria, Cluster Mode con 3 shards Multi-AZ
- **CoreBankingAdapter:** fuente de verdad cuando el caché expira
- **Cache Invalidator:** escucha eventos de actualización del Core (webhook) para purgar claves
- **Key Convention:** bp:{userId}:{dataType}:{version}

12 Alta Disponibilidad, Tolerancia a Fallos y DR

99.95%

SLA Target

< 1h

RPO

< 4h

RTO

2 AZ

Mínimo por
región

Estrategias de HA

EKS Multi-AZ: Pods distribuidos en 3 AZs de us-east-1; HPA (Horizontal Pod Autoscaler) basado en CPU/RPS

RDS Multi-AZ: Failover automático en < 120s; réplicas de lectura para consultas de movimientos

ElastiCache Redis Cluster: 3 shards, 2 réplicas por shard, failover automático

Circuit Breaker (Resilience4j): Threshold 50% errores en 10s → estado OPEN; half-open cada 30s

Retry + Exponential Backoff: 3 intentos con jitter para llamadas al Core Banking

Estrategia DR (Recuperación)

Región secundaria sa-east-1 (Brasil): Modo Warm Standby — réplicas activas, tráfico enrutado por Route53 Health Checks

RDS Cross-Region Read Replica: Replicación async a sa-east-1; promovida a primary en DR

S3 Cross-Region Replication: Backups, artefactos y logs replicados automáticamente

Runbook automatizado: AWS Systems Manager Runbook ejecuta el failover en secuencia verificada

Auto-healing: Kubernetes liveness/readiness probes reinician pods en fallo; ASG reemplaza instancias EC2

13 Infraestructura Cloud — AWS

La infraestructura se despliega en AWS con una VPC Multi-AZ en **us-east-1** (producción) y **sa-east-1** (DR/Warm Standby), conectadas vía **AWS Transit Gateway**. Las subredes privadas alojan el backend; las públicas el Application Load Balancer y CloudFront.

Cómputo y Orquestación

Amazon EKS (Kubernetes 1.30)
AWS Fargate (serverless pods)
AWS Lambda (Onboarding/Notif.)
AWS Auto Scaling Groups

Datos y Mensajería

Amazon RDS PostgreSQL Multi-AZ
Amazon ElastiCache Redis 7
Amazon SQS FIFO + DLQ
Amazon SNS (fan-out notif.)
Amazon S3 (artefactos / logs)

Seguridad y Red

AWS WAF v2 + Shield Standard
AWS KMS (cifrado en reposo)
AWS Secrets Manager
AWS GuardDuty (IDS/IPS)
Amazon CloudFront (CDN + TLS)
AWS Route53 (DNS + Health Checks)

APIs e Integración

Amazon API Gateway (REST + HTTP)
AWS AppMesh (service mesh mTLS)
AWS Private Link (Core Banking)
Amazon EventBridge (scheduling)

IA / ML Especializados

Amazon Rekognition (facial)
Amazon Textract (OCR docs)
Amazon Fraud Detector
Amazon Comprehend (análisis texto)

CI/CD y Delivery

AWS CodePipeline + CodeBuild
Amazon ECR (Container Registry)
AWS Systems Manager (Runbooks)
AWS Config (compliance drift)

12 Monitoreo y Observabilidad

Stack de Observabilidad

- AWS CloudWatch:** Métricas, logs, alarmas dashboards operativos en tiempo real
- AWS X-Ray:** Distributed tracing de cada request desde API GW hasta base de datos
- Amazon Managed Grafana:** Dashboards ejecutivos con métricas de negocio y técnicas
- AWS GuardDuty:** Detección de amenazas y comportamiento anómalo
- Alertas SNS:** Notificación a equipos de operaciones vía email/Slack/PagerDuty

Métricas Clave (SLOs)

Métrica	Umbral
Latencia API P99	< 800 ms
Tasa de error API	< 0.1%
Cache hit ratio	> 85%
Auth token latency	< 300 ms
Transfer success rate	> 99.5%
DB Query P95	< 100 ms